

Esercizi del Capitolo 1

Perché gli oggetti

LEGGI PRIMA DI COMINCIARE

Questi tre esercizi usano **soltanto** quello che hai visto nel capitolo 1: dizionari, funzioni e ragionamento. Non ti serve ancora saper scrivere `class`: quello arriva con gli esercizi del capitolo 2, che riprenderanno proprio il lavoro che farai qui nel terzo esercizio.

Dove lavorare

Se usi un **notebook**, crea una cella nuova per ogni esercizio e premi `Shift + Invio`. Se usi **PyCharm**, crea un file nuovo per ogni esercizio (per esempio `esercizio_1.py`) e premi il pulsante verde *Run*.

Come sono fatte le tracce

Ogni esercizio ti dà tutto quello che serve: la situazione di partenza, l'elenco preciso di che cosa devi fare, il codice da cui partire, il blocco di prova da incollare in fondo e l'**output esatto** che il tuo programma deve produrre.

Quell'output è il tuo strumento di autocorrezione: se quello che ottieni combacia riga per riga con quello scritto nella traccia, l'esercizio è corretto. Se non combacia, la differenza ti dice esattamente dove guardare.

Difficoltà	Che cosa aspettarti	Tempo indicativo
● ● ● Guidato	Ti si dice quasi tutto, devi solo metterlo insieme.	15 – 20 minuti
● ● ● Autonomo	Sai cosa serve, devi scriverlo tu.	30 – 40 minuti
● ● ● Impegnativo	Devi anche decidere <i>come</i> impostare il lavoro.	40 – 60 minuti

UN CONSIGLIO

Fai gli esercizi **in ordine**: il secondo riprende il primo e il terzo riprende il secondo. Se ti blocchi, non saltare all'esercizio dopo: rileggi il paragrafo del capitolo indicato nei suggerimenti.

Caccia agli oggetti

La situazione

Nel capitolo 1 hai scoperto una cosa che forse ti è sfuggita finora: in Python **tutto è un oggetto**. Una stringa, una lista e un dizionario sono oggetti, e quando scrivi `parola.upper()` stai chiamando un *metodo* su un oggetto, esattamente come farai fra poco con le classi scritte da te.

In questo esercizio devi far parlare gli oggetti che già usi tutti i giorni, trovando il metodo giusto per ogni richiesta.

Codice di partenza

Copia queste tre righe: sono l'inizio del tuo programma e **non vanno modificate**.

```
parola = " Data Engineer "  
numeri = [5, 3, 9, 1]  
listino = {"mouse": 25.0, "tastiera": 45.0}
```

Che cosa devi fare

Parte A. Sotto quelle tre righe, scrivi le istruzioni `print(...)` che stampano, **in quest'ordine esatto**:

1. la classe di `parola`, poi quella di `numeri`, poi quella di `listino` (tre righe separate);
2. il contenuto di `parola` **senza gli spazi** all'inizio e alla fine;
3. la stessa cosa del punto 2, ma **tutta in maiuscolo**;
4. quante volte compare la lettera `"a"` minuscola dentro `parola`;
5. la lista `numeri` **ordinata** dal più piccolo al più grande;
6. l'elenco dei **nomi dei prodotti** presenti nel listino (solo i nomi, non i prezzi);
7. il **prezzo del mouse**.

Regola dell'esercizio: **non devi scrivere nessuna funzione tua**. Ogni riga si risolve chiamando un metodo dell'oggetto, oppure usando una funzione già pronta di Python.

Parte B. Nella stessa cella (o in fondo al file), rispondi per iscritto in un commento a queste tre domande:

1. Nella riga `parola.strip()` : qual è l'**oggetto** e qual è il **metodo**?
2. Dopo aver eseguito `parola.strip()`, la variabile `parola` contiene ancora gli spazi oppure no? Verificalo stampandola di nuovo e spiega il risultato.
3. Per ordinare la lista hai usato un metodo di `numeri` oppure una funzione che riceve `numeri`? Che differenza c'è fra le due scritture?

Output atteso

Il tuo programma, nella parte A, deve stampare **esattamente** questo:

```
<class 'str'>
<class 'list'>
<class 'dict'>
Data Engineer
DATA ENGINEER
2
[1, 3, 5, 9]
['mouse', 'tastiera']
25.0
```

SUGGERIMENTI

- Per sapere quali metodi ha un oggetto, stampa `dir(parola)` e guarda i nomi che **non** iniziano con i doppi trattini bassi.
- Il metodo che toglie gli spazi ai lati di una stringa inizia per `s`. Quello che conta le occorrenze di un carattere inizia per `c`.
- Per ordinare una lista hai due strade: il metodo `.sort()`, che modifica la lista stessa e non restituisce niente, oppure la funzione `sorted(...)`, che restituisce una lista nuova. Vanno bene entrambe, ma attenzione: se usi `.sort()` devi stampare la lista *dopo*, in una riga separata.
- Per l'elenco dei nomi del listino, cerca il metodo dei dizionari che restituisce le chiavi, e ricordati di trasformarlo in lista con `list(...)` per ottenere le parentesi quadre dell'output atteso.
- Se ti serve ripassare, rileggi il paragrafo 1.4 del capitolo e la prova 4.

HAI FINITO QUANDO

- ☐ il tuo output combacia riga per riga con quello atteso, comprese le parentesi e le virgolette;
- ☐ non hai scritto nessuna funzione con `def` ;
- ☐ hai risposto per iscritto alle tre domande della parte B.

Il magazzino senza classi

La situazione

Un piccolo negozio di informatica ti chiede un programma per tenere il magazzino. Ogni prodotto ha un nome, un prezzo e la quantità disponibile. Per ora lo scriverai **come si faceva prima delle classi**: i dati nei dizionari e le operazioni nelle funzioni.

Il programma funzionerà benissimo. Nell'esercizio 3 scoprirai perché conviene comunque cambiare approccio.

Codice di partenza

Parti da questo. Le tre variabili e la funzione `descrivi` ti sono già date.

```
prodotto1 = {"nome": "Mouse", "prezzo": 25.0, "quantita": 4}
prodotto2 = {"nome": "Tastiera", "prezzo": 45.0, "quantita": 2}
prodotto3 = {"nome": "Monitor", "prezzo": 180.0, "quantita": 0}

def descrivi(p):
    return p["nome"] + " - " + str(p["prezzo"]) + " EUR x " + str(p["quantita"])
```

Che cosa devi fare

Scrivi **quattro funzioni**, rispettando esattamente questi nomi e questi comportamenti.

Funzione	Che cosa deve fare
<code>valore_totale(p)</code>	Restituisce quanto vale in tutto quel prodotto in magazzino, cioè il prezzo moltiplicato per la quantità.
<code>disponibile(p)</code>	Restituisce <code>True</code> se del prodotto c'è almeno un pezzo, <code>False</code> se la quantità è zero.
<code>vendi(p, quanti)</code>	Vende <code>quanti</code> pezzi del prodotto. Se ce ne sono abbastanza, diminuisce la quantità e restituisce <code>True</code> . Se non ce ne sono abbastanza non deve modificare nulla : stampa il messaggio <code>non ci sono abbastanza pezzi di</code> seguito dal nome del prodotto, e restituisce <code>False</code> .
<code>valore_magazzino(lista)</code>	Riceve una <i>lista</i> di prodotti e restituisce la somma dei loro valori totali. Deve riusare <code>valore_totale</code> , non rifare il calcolo.

Blocco di prova

Incolla questo blocco **alla fine** del tuo programma, senza modificarlo. Serve a verificare che tutto funzioni.

```
magazzino = [prodotto1, prodotto2, prodotto3]

print(descrivi(prodotto1))
print("valore del Mouse:", valore_totale(prodotto1))
print("Monitor disponibile?", disponibile(prodotto3))
print("valore del magazzino:", valore_magazzino(magazzino))

print("vendo 2 Mouse:", vendi(prodotto1, 2))
print(descrivi(prodotto1))

print("vendo 5 Tastiere:", vendi(prodotto2, 5))
print(descrivi(prodotto2))

print("valore del magazzino alla fine:", valore_magazzino(magazzino))
```

Output atteso

```
Mouse - 25.0 EUR x 4
valore del Mouse: 100.0
Monitor disponibile? False
valore del magazzino: 190.0
vendo 2 Mouse: True
Mouse - 25.0 EUR x 2
non ci sono abbastanza pezzi di Tastiera
vendo 5 Tastiere: False
Tastiera - 45.0 EUR x 2
valore del magazzino alla fine: 140.0
```

ATTENZIONE A DUE PUNTI

- Il messaggio `non ci sono abbastanza pezzi di Tastiera` compare **prima** della riga `vendo 5 Tastiere: False`. Non è un errore: la funzione stampa il messaggio mentre viene eseguita, e solo dopo il `print` esterno mostra il valore restituito.
- Dopo la vendita fallita la Tastiera deve avere ancora **2 pezzi**. Se ne ha meno, la tua funzione ha modificato la quantità prima di controllare: sposta il controllo all'inizio.

SUGGERIMENTI

- Per leggere un dato da un dizionario si scrive `p["prezzo"]` ; per modificarlo, `p["quantita"] = ...`.
- In `vendi` fai prima il controllo «ne ho abbastanza?» e solo dopo la sottrazione.
- In `valore_magazzino` parti da una variabile `totale = 0` , poi scorri la lista con un ciclo `for` e a ogni giro somma il valore del prodotto corrente.
- Le funzioni devono restituire il valore con `return` , non stamparlo: è il blocco di prova che si occupa di stampare. L'unica funzione che stampa qualcosa per conto suo è `vendi` , e solo nel caso di errore.

HAI FINITO QUANDO

- ☐ il tuo output combacia riga per riga con quello atteso;
- ☐ hai scritto tutte e quattro le funzioni con i nomi indicati;
- ☐ `valore_magazzino` chiama `valore_totale` invece di ripetere la moltiplicazione;
- ☐ dopo la vendita fallita la quantità della Tastiera è rimasta 2.

Progetta la classe Prodotto

La situazione

Il programma dell'esercizio 2 funziona, ma ha esattamente i difetti descritti nel capitolo 1: i dati stanno nei dizionari, le funzioni stanno per conto loro, e niente impedisce a chiunque di rovinare un prodotto.

In questo esercizio **non scriverai codice funzionante**. Farai il lavoro che viene prima: decidere com'è fatta la classe. È quello che fa davvero un programmatore prima di aprire l'editor. Il risultato di questo esercizio lo userai nell'**esercizio 3 del capitolo 2**, dove la classe la scriverai per davvero.

CHE COSA CONSEGNI

Un testo, non un programma. Puoi scriverlo in una cella di testo del notebook, in un commento lungo dentro un file `.py`, oppure su un foglio. Deve contenere le quattro parti descritte qui sotto.

Parte A — Trova i difetti

Prendi il programma dell'esercizio 2 e scrivi **tre righe di codice** che lo mettono in una situazione impossibile *senza che Python protesti*. Per ognuna indica: la riga, che cosa succede subito dopo averla eseguita, e quando e dove salterà fuori il problema.

Il tipo di cose da cercare: un prezzo negativo, una quantità che diventa testo, il nome di una chiave scritto male, un prodotto a cui manca un dato.

Provale davvero prima di scriverle: alcune daranno un errore subito, altre — le più pericolose — non ne daranno nessuno.

Parte B — Progetta la classe

Compila questo schema. Il nome della classe è già deciso: `Prodotto`.

Campi (i dati)	Tipo	A che cosa serve

Metodi (le operazioni)	Che cosa riceve	Che cosa restituisce	Che cosa fa

Infine elenca le **regole che devono valere sempre**, qualunque cosa succeda. Scrivine almeno tre, in italiano, una per riga. Esempio di come si scrivono: «*la quantità non può mai essere negativa*».

Parte C — Riscrivi le chiamate

Queste sono cinque righe del programma dell'esercizio 2. Per ognuna scrivi come diventerebbe **usando la classe** che hai appena progettato. Non devi implementare niente: devi solo mostrare come cambierebbe la scrittura.

Esempio già svolto: `valore_totale(prodotto1)` diventa `mouse.valore_totale()`

```
1) prodotto1 = {"nome": "Mouse", "prezzo": 25.0, "quantita": 4}
2) descrivi(prodotto1)
3) disponibile(prodotto3)
4) vendi(prodotto1, 2)
5) prodotto1["quantita"]
```

Parte D — Una domanda di ragionamento

Nel programma dell'esercizio 2 c'è una funzione diversa dalle altre: `valore_magazzino(lista)`, che non lavora su un singolo prodotto ma su una lista di prodotti.

Secondo te dovrebbe diventare un metodo della classe `Prodotto`? Rispondi in quattro o cinque righe, motivando. Se la risposta è no, prova a dire **di chi** dovrebbe essere quel compito.

Per ragionare, chiediti se la frase «*un prodotto sa calcolare il valore di tutto il magazzino*» ha senso, oppure se descrive la responsabilità di qualcun altro.

SUGGERIMENTI

- Per i campi, guarda quali chiavi hanno i dizionari dell'esercizio 2: sono quelli.
- Per i metodi, guarda i nomi delle funzioni che hai scritto e chiediti quali ricevono un singolo prodotto come primo argomento: quelle diventano metodi.
- Il paragrafo 1.4 del capitolo spiega la differenza fra campi e metodi; la tabella del paragrafo 1.3 mostra un esempio già svolto con lo studente.
- Per le regole che devono valere sempre, rileggi la parte A: ogni difetto che hai trovato corrisponde a una regola che nessuno stava facendo rispettare.

HAI FINITO QUANDO

- ☐ hai trovato tre modi di rovinare i dati e per ognuno hai detto *quando* si manifesta il problema;
- ☐ lo schema della classe ha almeno tre campi e quattro metodi, ognuno con il suo nome preciso;
- ☐ hai scritto almeno tre regole che devono valere sempre;
- ☐ hai riscritto tutte e cinque le chiamate della parte C;
- ☐ hai risposto alla domanda della parte D motivando la scelta.